

Convolutional Networks

Convolution Filters: What They Learn

Introduction to Deep Learning · Class 25

Outline

Stacking Filters

A Standard Architecture

What a Filter Responds To

Reading and Fooling a Network

Style Transfer

Putting It Together

Stacking Filters

Opening the trunk

Class 23 defined one layer. Class 24 built three tasks on a trunk it treated as a given. Today the trunk itself: how filters compose, what a standard one looks like, and what the filters turn out to detect.

Question	Tool	Where
how do small filters make big ones?	composition, receptive fields	§10.2.7–10.2.8
what does a standard trunk look like?	VGG-16, layer by layer	§10.2.8
what does a filter respond to?	the weights, maximising a unit, the gradient	§10.3
can the gradient be turned against it?	adversarial attacks, DeepDream	§10.3.4–10.3.5
what is style, to a network?	the style matrix	§10.6

One tool does most of the work today: the gradient of something the network computes, taken with respect to its *input*.

Notation for today

M	kernel size, $M \times M$	$a^{(c)}$	pre-activation of output unit c
$C_{\text{in}}, C_{\text{out}}$	channels in and out of a layer	$\mathbf{x}$	the input image, as pixels
$\mathbf{W}_l$	the kernel of layer l	$E(\mathbf{x}, t)$	the error on $\mathbf{x}$ with true label t
a_{ijk}	pre-activation at (i, j) of channel k	$\mathbf{F}$	the style matrix, $K \times K$

- Bishop writes the filter tensor as $M \times M \times C_{\text{IN}} \times C_{\text{OUT}}$; Class 23 called the same object Ω and the kernel size k
- Bishop reserves *depth* for the number of layers; the number of channels is never called depth here

Bishop & Bishop (2024), §10.2.7, §10.3.

A layer, counted

Definition 1 (A convolutional layer with channels)

A layer with C_{in} input channels and C_{out} output channels, kernel $M \times M$, is a filter tensor of size $M \times M \times C_{\text{in}} \times C_{\text{out}}$ with

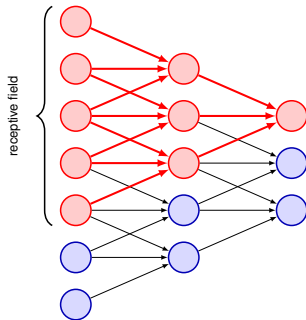
$$(M^2 C_{\text{in}} + 1) C_{\text{out}}$$

independent weights and biases. Its C_{out} output channels are the input channels of the next layer, as the RGB channels were of the first.

- Stack such layers, with optional pooling between, and the effective receptive field of a unit grows with every layer: Class 23 measured it
- The final fully connected layers see the whole image and may hold most of the *parameters*, even though most of the *connections* are convolutional

Bishop & Bishop (2024), §10.2.7, Fig. 10.9.

Reach grows with depth



The red unit at the top reads three units of the middle layer, each of which reads three of the first, so it depends on five inputs: the effective receptive field grows with every layer, and a stack of small kernels reaches as far as one large kernel for fewer parameters. Bishop & Bishop (2024), Fig. 10.9.

Two small filters are one large one — until a ReLU

Proposition 2 (Composition of kernels)

*Two convolutions in succession with kernels $\mathbf{W}_1 (k_1 \times k_1)$ and $\mathbf{W}_2 (k_2 \times k_2)$, with no nonlinearity between them, equal one convolution with a kernel of size $(k_1 + k_2 - 1)^2$, namely $\mathbf{W}_2 * \mathbf{W}_1$. With a ReLU between them they are, in general, no single kernel at all.*

- **Proof.** Both operations are linear and translation-equivariant (Class 23), so their composition is too, and by the converse there it is a convolution; its kernel is the response to a single one, which is $\mathbf{W}_2 * \mathbf{W}_1$. □
- So the stack of (a) is not only cheaper than the big kernel: it is **constrained** to big kernels that factor into small ones — an inductive bias, in Class 18's sense.
- The ReLU between layers is what makes depth more than a factorisation: the second filter reads a rectified map, and the whole is not linear in the image.

Bishop & Bishop (2024), §10.2.8; Class 23, the equivariance theorem.

Two small filters are one large one — until a ReLU

Proposition 2 (Composition of kernels)

*Two convolutions in succession with kernels $\mathbf{W}_1$ ($k_1 \times k_1$) and $\mathbf{W}_2$ ($k_2 \times k_2$), with no nonlinearity between them, equal one convolution with a kernel of size $(k_1 + k_2 - 1)^2$, namely $\mathbf{W}_2 * \mathbf{W}_1$. With a ReLU between them they are, in general, no single kernel at all.*

- Proof. Both operations are linear and translation-equivariant (Class 23), so their composition is too, and by the converse there it is a convolution; its kernel is the response to a single one, which is $\mathbf{W}_2 * \mathbf{W}_1$. □
- **So the stack of (a) is not only cheaper than the big kernel: it is constrained to big kernels that factor into small ones — an inductive bias, in Class 18's sense.**
- The ReLU between layers is what makes depth more than a factorisation: the second filter reads a rectified map, and the whole is not linear in the image.

Bishop & Bishop (2024), §10.2.8; Class 23, the equivariance theorem.

Two small filters are one large one — until a ReLU

Proposition 2 (Composition of kernels)

*Two convolutions in succession with kernels $\mathbf{W}_1$ ($k_1 \times k_1$) and $\mathbf{W}_2$ ($k_2 \times k_2$), with no nonlinearity between them, equal one convolution with a kernel of size $(k_1 + k_2 - 1)^2$, namely $\mathbf{W}_2 * \mathbf{W}_1$. With a ReLU between them they are, in general, no single kernel at all.*

- Proof. Both operations are linear and translation-equivariant (Class 23), so their composition is too, and by the converse there it is a convolution; its kernel is the response to a single one, which is $\mathbf{W}_2 * \mathbf{W}_1$. □
- So the stack of (a) is not only cheaper than the big kernel: it is **constrained** to big kernels that factor into small ones — an inductive bias, in Class 18's sense.
- **The ReLU between layers is what makes depth more than a factorisation: the second filter reads a rectified map, and the whole is not linear in the image.**

Bishop & Bishop (2024), §10.2.8; Class 23, the equivariance theorem.

The 1×1 convolution

Definition 3 (1×1 convolution)

A layer with $M = 1$: every output channel at position (i, j) is a weighted sum of all input channels at that position, plus a bias, through the activation function,

$$z_{ijc'} = h\left(\sum_c w_{cc'} x_{ijc} + b_{c'}\right),$$

with $C_{\text{in}}C_{\text{out}} + C_{\text{out}}$ parameters and no spatial mixing at all.

- It changes the number of channels without touching the resolution: the same linear map applied at every pixel
- Class 24 used it at the end of a segmentation network to turn feature channels into C class scores; it is also how a trunk is made narrower between expensive layers

Prince (2023), §10.5.1, Fig. 10.14.

A Standard Architecture

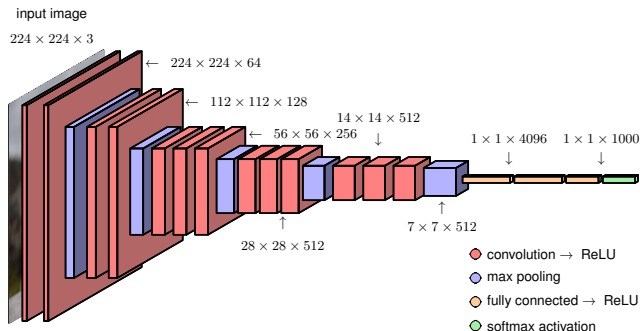
VGG-16, layer by layer

Stage	Layers	Output	Note
input	—	$224 \times 224 \times 3$	RGB
block 1	2× conv 3×3 , 64 ch; pool	$112 \times 112 \times 64$	first layer: 28 parameters per filter
block 2	2× conv, 128 ch; pool	$56 \times 56 \times 128$	channels double as the side halves
blocks 3–5	3× conv each, 256/512/512 ch; pool	$7 \times 7 \times 512$	25,088 units at the end
head	FC 4096, FC 4096, FC 1000	1000	softmax at the end

- Three design rules and almost no other choices: every kernel 3×3 , stride 1, same padding, ReLU; every pooling 2×2 , stride 2; channels doubled after each pooling so the representation does not shrink too fast
- 16 learnable layers; roughly 138 million parameters, nearly 103 million of them in the first fully connected layer

Bishop & Bishop (2024), §10.2.8, Fig. 10.10; Simonyan & Zisserman (2015).

VGG-16, drawn



Five blocks of 3×3 convolutions and 2×2 pooling to $7 \times 7 \times 512$, then three fully connected layers and a softmax; 138 million parameters, nearly 103 million in the first fully connected layer. Bishop & Bishop (2024),

Fig. 10.10.

What ImageNet did to the field

- Convolutional networks were the first deep networks to work in applications: LeNet read handwritten digits in 1989
- ImageNet: some 14 million labelled images in nearly 22,000 categories; the challenge subset has 1,000 and random guessing would score 99.9% error
- Top-5 error: about 25.5% before; 15.3% for AlexNet in 2012, with ReLUs, GPUs and dropout (Prince reports 16.4%); about 3% later, past the 5% or so of humans
- The lesson Bishop draws is not only depth: large data and a model with the **right inductive bias** together

The trunk is a hypothesis about images, and ImageNet was large enough to reward it.

Bishop & Bishop (2024), §10.2.8; Prince (2023), §10.5.1.

What a Filter Responds To

The visual cortex

- Hubel and Wiesel (1959) recorded single neurons in the visual cortex of cats while showing them stimuli
- **Simple cells** fire for an edge at one orientation and one position, and little else; their response is modelled by a Gabor filter
- **Complex cells** combine simple cells and are partly invariant to small shifts — the role pooling plays in a convolutional network
- Deeper cells are more specific and more invariant still; the notional *grandmother cell* fires for one person at any location, scale or lighting
- The neocognitron (Fukushima, 1980) copied this: local receptive fields, shared weights, local pooling — but no end-to-end training, since it predated backpropagation

Bishop & Bishop (2024), §10.3.1.

The Gabor filter

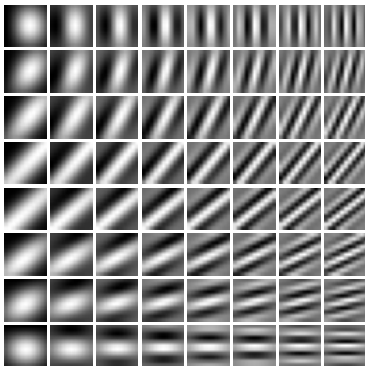
Definition 4 (Gabor filter)

$$G(x, y) = A \exp(-\alpha \tilde{x}^2 - \beta \tilde{y}^2) \sin(\omega \tilde{x} + \phi),$$
$$\begin{aligned}\tilde{x} &= (x - x_0) \cos \theta + (y - y_0) \sin \theta, \\ \tilde{y} &= -(x - x_0) \sin \theta + (y - y_0) \cos \theta.\end{aligned}$$

- $(\tilde{x}, \tilde{y})$ is the coordinate system rotated by θ , so the sine is a spatial oscillation in the direction θ with frequency ω and phase ϕ
- The exponential is a decay envelope that localises the filter around (x_0, y_0) , with rates α and β
- Correlate an image with it and the response is large where there is an edge or a stripe at that orientation and scale, which is what a simple cell does

Bishop & Bishop (2024), Eqs. 10.6–10.8, Fig. 10.11.

Gabor filters, drawn



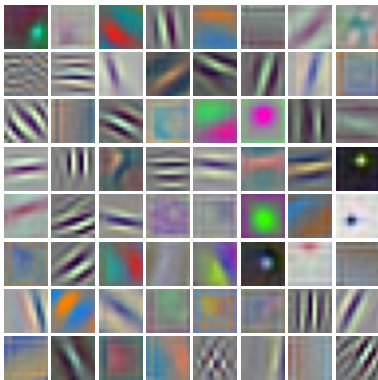
Eq. 10.6: the orientation θ runs from 0 in the top row to $\pi/2$ in the bottom row, the frequency ω from 1 in the left column to 10 in the right. Bishop & Bishop (2024), Fig. 10.11.

Reading the first layer directly

- A first-layer filter is a small patch in pixel space, so its weights can be shown as an image; the unit fires when the inner product with the patch under it is large
- The filters a network learns on ImageNet look remarkably like Gabor filters
- That is *not* evidence that the network is a model of the brain: many statistical methods recover the same filters from natural images. They are a property of the **statistics of images**, useful to any system that looks at them
- Deeper layers cannot be read this way: their inputs are filter responses, not pixels

Bishop & Bishop (2024), §10.3.2, Fig. 10.12; Hyvärinen et al. (2009).

The first layer of AlexNet



Learned first-layer filters of AlexNet: oriented edges and stripes at several scales, and colour-opponent blobs — many of them Gabor filters to the eye. Bishop & Bishop (2024), Fig. 10.12.

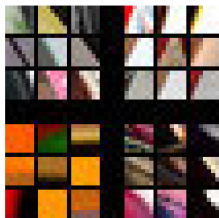
Deeper layers: ask what excites a unit

- Hubel and Wiesel's method, applied to a network: show it many image patches and keep the ones that excite a given unit most
- Doing this for a five-layer network trained on ImageNet shows a **hierarchy**: edges in layer 1, textures and simple shapes in layer 2, parts of objects in layer 3, whole objects in layer 5
- The same locality and pooling that gave Class 23's growing receptive field give this growing meaning

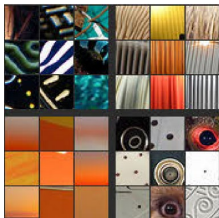
Depth composes filters into detectors of larger and more abstract things, layer by layer — what the composition proposition permits, and the ReLU makes non-trivial.

Bishop & Bishop (2024), §10.3.2, Fig. 10.13; Zeiler & Fergus (2013).

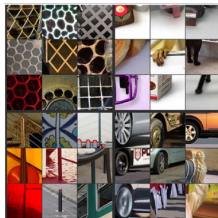
Patches that excite units, by layer



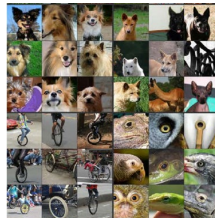
Layer 1



Layer 2



Layer 3



Layer 5

For four channels in each of layers 1, 2, 3 and 5, the nine validation patches with the strongest activation, as a 3×3 grid each: edges, then textures, then object parts, then objects. Bishop & Bishop (2024), Fig. 10.13, from Zeiler & Fergus (2013).

Or optimise the image

Proposition 5 (Maximise the pre-activation, not the probability)

To find the image most representative of class c , ascend the pre-activation $a^{(c)}(\mathbf{x})$, not the softmax output p_c :

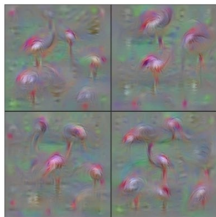
$$\frac{\partial \log p_c}{\partial a^{(c')}} = \delta_{cc'} - p_{c'},$$

so the gradient of p_c depends on every other class through the denominator: it would make the image less like a cat as much as more like a dog.

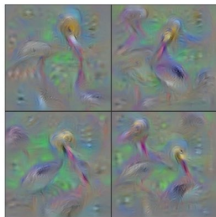
- Unconstrained ascent sends pixels to infinity, so it is regularised: a sum-of-squares penalty, blurring between steps, clipping of pixels that contribute little
- The result is a **synthetic image** of the class; the same gradient, pointed the other way, is an attack

Bishop & Bishop (2024), §10.3.2, Fig. 10.14; Yosinski et al. (2015).

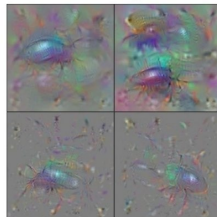
Synthetic images of four classes



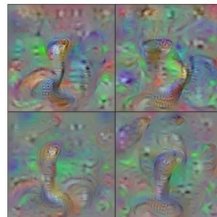
Flamingo



Pelican



Ground Beetle



Indian Cobra

Images that maximise a class output of a trained classifier, four regularisation settings per class. Bishop & Bishop (2024), Fig. 10.14, from Yosinski et al. (2015).

Reading and Fooling a Network

Saliency: which pixels decided

Definition 6 (Grad-CAM)

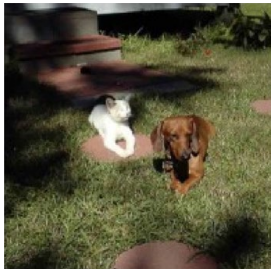
For an input image and a class c , take the last convolutional layer, with M_k units in channel k and pre-activations $a_{ij}^{(k)}$. Average the gradient of the class pre-activation over each channel, and weight the channels by it:

$$\alpha_k = \frac{1}{M_k} \sum_i \sum_j \frac{\partial a^{(c)}}{\partial a_{ij}^{(k)}}, \quad \mathbf{L} = \sum_k \alpha_k \mathbf{A}^{(k)}.$$

- The last convolutional layer is the right place: it still has positions, which the fully connected layers lose, and the most semantic features
- $\mathbf{L}$ has the size of that layer — 14×14 for VGG-16 — and is drawn over the image as a heat map

Bishop & Bishop (2024), §10.3.3, Eqs. 10.9–10.10; Selvaraju et al. (2016).

Saliency, on an image



Original image



Saliency map for 'dog'



Saliency map for 'cat'

Grad-CAM for VGG-16: the same image, and the regions that decide 'dog' and 'cat' respectively. Bishop & Bishop (2024), Fig. 10.15, from Selvaraju et al. (2016).

Turning the gradient against the network

Definition 7 (Fast gradient sign method)

With the network fixed and $E(\mathbf{x}, t)$ its error on image $\mathbf{x}$ with true label t , move every pixel by ϵ in the direction that increases the error:

$$\mathbf{x}' = \mathbf{x} + \epsilon \operatorname{sign}(\nabla_{\mathbf{x}} E(\mathbf{x}, t)).$$

Proposition 8 (Why a tiny ϵ is enough)

For an activation linear in the pixels, $a = \mathbf{w}^T \mathbf{x} + b$, the attack changes it by $\epsilon \|\mathbf{w}\|_1$: one step of ϵ per pixel, added over all d of them, so it grows like d while no single pixel changes by more than ϵ . Training moves the parameters to reduce E ; the attack moves the image to increase it, by the same backpropagation.

Bishop & Bishop (2024), §10.3.4, Eq. 10.11; Goodfellow et al. (2014).

What the attacks show

- An imperceptible change makes a network misclassify with high confidence
- Not over-fitting: an image adapted to fool one network fools others, and a **linear** model is fooled the same way — Proposition 8 is the reason
- Physical objects can be built that a plain photograph of misclassifies; simple attacks yield to simple changes in training, sophisticated ones are an open problem



Left: panda 57.7% $\rightarrow$ gibbon 99.3%. Right: stop signs read as 45 mph.
Eykholt et al. (2018).

Bishop & Bishop (2024), Figs. 10.16–10.17, from Goodfellow et al. (2014) and

DeepDream: amplify what a layer already sees

Definition 9 (DeepDream)

Forward-propagate an image I to a chosen layer, set each unit's δ there equal to its own pre-activation, backpropagate to the pixels, and step the image along the gradient. This ascends

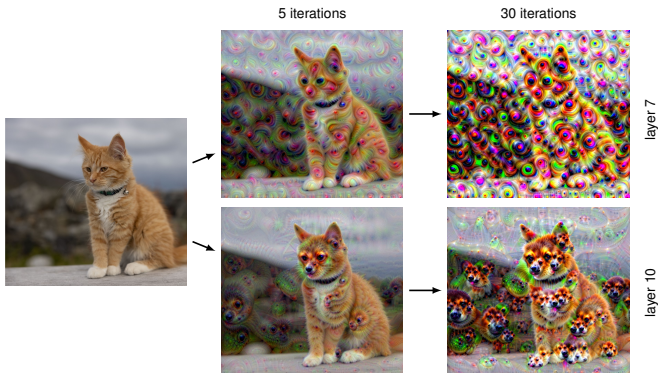
$$F(I) = \sum_{i,j,k} a_{ijk}(I)^2,$$

the sum of squared pre-activations of the layer.

- Whatever the layer responds to a little is made to respond a lot: a cat-like cloud becomes more cat-like, over several iterations, with smoothing and clipping to keep it plausible
- From noise it makes an image entirely of the network's own

Bishop & Bishop (2024), §10.3.5, Eq. 10.12, Exercise 10.10; Mordvintsev et al. (2015).

DeepDream, on an image



Layer 7 (top) and layer 10 (bottom) of VGG-16, after five and thirty iterations. Bishop & Bishop (2024), Fig. 10.18.

Style Transfer

Content and style, as errors

Definition 10 (Neural style transfer)

Generate G from a content image C and a style image S by descent on $E(G) = E_{\text{content}}(G, C) + E_{\text{style}}(G, S)$, at a convolutional layer with $I \times J$ maps and K channels:

$$E_{\text{content}} = \sum_{i,j,k} \{a_{ijk}(G) - a_{ijk}(C)\}^2, \quad F_{kk'} = \sum_{i,j} a_{ijk} a_{ijk'}, \quad E_{\text{style}} = \frac{\sum_{k,k'} \{F_{kk'}(G) - F_{kk'}(S)\}^2}{(2IJK)^2}.$$

- Content is matched *feature by feature, in place*; style through the $K \times K$ **style matrix** of co-occurrences, summed over every location; in practice $\sum_l \lambda_l E_{\text{style}}^{(l)}$ over several layers

Bishop & Bishop (2024), §10.6, Eqs. 10.13–10.17; Gatys et al. (2015).

Why the style matrix forgets position

Proposition 11 (The style matrix is translation-invariant)

Translating every feature map by the same amount leaves $F_{kk'} = \sum_{ij} a_{ijk} a_{ijk'}$ unchanged, up to the values that leave the frame, since the sum runs over all positions and a translation only permutes them. The content error has no such invariance: it compares position with position.

- **Proof.** Write $a'_{ijk} = a_{i+u, j+v, k}$. Then $\sum_{ij} a'_{ijk} a'_{ijk'} = \sum_{ij} a_{i+u, j+v, k} a_{i+u, j+v, k'}$, a re-indexing of the same sum over the positions that stay inside. □
- So F measures whether a vertical edge tends to come with an orange blob, *wherever* the two are — which is what one means by the style of a painting.
- Early layers match edges; later layers match larger structures. The layer chosen decides how literal the copy is.

Bishop & Bishop (2024), §10.6.

Why the style matrix forgets position

Proposition 11 (The style matrix is translation-invariant)

Translating every feature map by the same amount leaves $F_{kk'} = \sum_{ij} a_{ijk} a_{ijk'}$ unchanged, up to the values that leave the frame, since the sum runs over all positions and a translation only permutes them. The content error has no such invariance: it compares position with position.

- Proof. Write $a'_{ijk} = a_{i+u, j+v, k}$. Then $\sum_{ij} a'_{ijk} a'_{ijk'} = \sum_{ij} a_{i+u, j+v, k} a_{i+u, j+v, k'}$, a re-indexing of the same sum over the positions that stay inside. □
- **So F measures whether a vertical edge tends to come with an orange blob, *wherever* the two are — which is what one means by the style of a painting.**
- Early layers match edges; later layers match larger structures. The layer chosen decides how literal the copy is.

Bishop & Bishop (2024), §10.6.

Why the style matrix forgets position

Proposition 11 (The style matrix is translation-invariant)

Translating every feature map by the same amount leaves $F_{kk'} = \sum_{ij} a_{ijk} a_{ijk'}$ unchanged, up to the values that leave the frame, since the sum runs over all positions and a translation only permutes them. The content error has no such invariance: it compares position with position.

- Proof. Write $a'_{ijk} = a_{i+u, j+v, k}$. Then $\sum_{ij} a'_{ijk} a'_{ijk'} = \sum_{ij} a_{i+u, j+v, k} a_{i+u, j+v, k'}$, a re-indexing of the same sum over the positions that stay inside. □
- So F measures whether a vertical edge tends to come with an orange blob, *wherever* the two are — which is what one means by the style of a painting.
- **Early layers match edges; later layers match larger structures. The layer chosen decides how literal the copy is.**

Bishop & Bishop (2024), §10.6.

Style transfer, on an image



A canal scene (left) rendered in the style of Turner's *The Wreck of a Transport Ship* (centre) and of van Gogh's *The Starry Night* (right); the style image is inset. Bishop & Bishop (2024), Fig. 10.32, from Gatys et al. (2015).

Putting It Together

One gradient, five uses

Use	Differentiate	With respect to	Then
training	the error	the parameters	descend
synthetic images	a class pre-activation	the pixels	ascend, regularised
saliency	a class pre-activation	the last conv layer	average, weight, overlay
adversarial attack	the error	the pixels	one signed step of ϵ
DeepDream	$\sum a_{ijk}^2$ at a layer	the pixels	ascend, repeat
style transfer	content + style error	the pixels of G	descend from noise

Backpropagation does not care which end of the network it stops at. Everything in the second half of today is Class 11's algorithm with the image, not the weights, as the variable.

Takeaways

1. A layer with channels has $(M^2 C_{\text{in}} + 1) C_{\text{out}}$ parameters; stacking small kernels reaches as far as a large one for far fewer (Definition 1)
2. Two linear kernels are one kernel; the ReLU between them is what makes depth more than a factorisation (Proposition 2)
3. Simple cells are Gabor filters (Definition 4), and so, to the eye, are a trained first layer — because of image statistics, not because the network is a brain
4. Deeper units are read by what excites them: edges, textures, parts, objects; or by ascending the pre-activation (Proposition 5)
5. Grad-CAM weights the last convolutional layer by averaged gradients (Definition 6)
6. One signed step of ϵ per pixel fools a network, and fools a linear model for a linear reason (Definition 7, Proposition 8)
7. Style is the co-occurrence of features summed over position, which is why the style matrix forgets where (Definition 10, Proposition 11)

Next: why the deepest trunks stop training well, and the residual connection that fixes it.

References i

1. C. M. Bishop & H. Bishop, *Deep Learning: Foundations and Concepts*, Springer, 2024. §10.2.7–10.2.8 (Figs. 10.9–10.10, Exercise 10.8); §10.3 (Eqs. 10.6–10.12, Figs. 10.11–10.18); §10.6 (Eqs. 10.13–10.17, Fig. 10.32).
2. S. J. D. Prince, *Understanding Deep Learning*, MIT Press, 2023. §10.5.1, Fig. 10.14.
3. D. H. Hubel & T. N. Wiesel, “Receptive fields of single neurones in the cat’s striate cortex”, *J. Physiology* 148, 1959.
4. K. Fukushima, “Neocognitron”, *Biological Cybernetics* 36, 1980.
5. Y. LeCun et al., “Backpropagation applied to handwritten zip code recognition”, *Neural Computation* 1(4), 1989.
6. A. Krizhevsky, I. Sutskever & G. E. Hinton, “ImageNet classification with deep convolutional neural networks”, NeurIPS 2012.
7. K. Simonyan & A. Zisserman, “Very deep convolutional networks for large-scale image recognition”, ICLR 2015.
8. A. Hyvärinen, J. Hurri & P. O. Hoyer, *Natural Image Statistics*, Springer, 2009.
9. M. D. Zeiler & R. Fergus, “Visualizing and understanding convolutional networks”, ECCV 2014 (arXiv 2013).

References ii

10. J. Yosinski et al., “Understanding neural networks through deep visualization”, ICML Deep Learning Workshop, 2015.
11. R. R. Selvaraju et al., “Grad-CAM: visual explanations from deep networks via gradient-based localization”, ICCV 2017 (arXiv 2016).
12. I. J. Goodfellow, J. Shlens & C. Szegedy, “Explaining and harnessing adversarial examples”, ICLR 2015 (arXiv 2014).
13. K. Eykholt et al., “Robust physical-world attacks on deep learning visual classification”, CVPR 2018.
14. A. Mordvintsev, C. Olah & M. Tyka, “Inceptionism: going deeper into neural networks”, Google Research Blog, 2015.
15. L. A. Gatys, A. S. Ecker & M. Bethge, “A neural algorithm of artistic style”, arXiv:1508.06576, 2015.

Questions?